EduTune AI

Instruction-Tuned AI Teaching Assistant
using Supervised Fine-Tuning, LoRA and QLoRA

Project Report & Presentation Guide

A complete explanation of the project — the idea, the theory, the dataset, the training, the evaluation, the application, and how to present and defend it in a viva.

Project at a glance	
Base model	Qwen/Qwen2.5-1.5B-Instruct
Method	Supervised fine-tuning (instruction tuning) with LoRA on a 4-bit base model = QLoRA
Trainable parameters	18,464,768 of 1,562,179,072 (1.18%)
Training data	840 instruction→response examples (756 train / 84 validation), 37 topics, 14 categories
Hardware	NVIDIA GeForce RTX 5050 Laptop GPU (8 GB) — training took 16.5 min, peak 2.81 GB
Key result	Validation perplexity 8.61 (base) → 2.48 (fine-tuned), a 71% reduction

Presented by: _____ Date: _____

Contents

1. Executive Summary . . . 4

1.1 Key numbers . . . 4

2. Problem Statement and Objectives . . . 5

2.1 Problem statement . . . 5

2.2 Objectives . . . 5

2.3 Scope . . . 5

3. Background Concepts . . . 6

3.1 Large language models and next-token prediction . . . 6

3.2 Pre-training vs fine-tuning . . . 6

3.3 Instruction tuning and supervised fine-tuning (SFT) . . . 6

3.4 PEFT — parameter-efficient fine-tuning . . . 6

3.5 LoRA — low-rank adaptation . . . 7

3.6 Quantisation and QLoRA . . . 7

3.7 Completion-only loss . . . 8

3.8 Perplexity — and why not “accuracy” . . . 8

4. System Architecture . . . 9

4.1 Inference architecture: one model, two behaviours . . . 9

5. Technology Stack . . . 10

6. Project Structure . . . 10

7. The Dataset . . . 12

7.1 Design approach . . . 12

7.2 Fields written for every topic . . . 12

7.3 Generated categories . . . 13

7.4 Example of a record that uses the input field . . . 13

7.5 Cleaning and splitting . . . 13

7.6 Instruction formatting . . . 14

8. Model and Training . . . 15

8.1 Choice of base model . . . 15

8.2 Training configuration . . . 15

8.3 Where the trainable parameters come from . . . 15

8.4 What the training script does, step by step . . . 16

8.5 Training results . . . 16

9. Evaluation . . . 18

9.1 Methodology . . . 18

9.2 Quantitative results . . . 18
9.3 Qualitative comparison (real outputs from results/comparison.json) . . . 20
9.4 Findings . . . 21
10. The Teaching Application . . . 22
10.1 Sidebar . . . 22
10.2 Pages . . . 22
10.3 Teaching modes . . . 22
10.4 Quick actions . . . 23
10.5 How Quiz, Viva and Interview use the fine-tuning . . . 23
10.6 Error handling . . . 23
11. Hardware and Performance . . . 24
12. Testing . . . 24
13. How to Install and Run . . . 24
14. Presentation Guide . . . 26
14.1 Suggested slide outline (about 12–15 minutes) . . . 26
14.2 Live demo script . . . 26
14.3 Numbers to remember . . . 27
15. Expected Viva Questions and Answers . . . 28
16. Limitations and Future Work . . . 30
16.1 Limitations . . . 30
16.2 Future work . . . 30
17. Conclusion . . . 30
Appendix A — Key Code . . . 31
A.1 4-bit quantisation (utils/model_loader.py) . . . 31
A.2 LoRA (training/train_qlora.py) . . . 31
A.3 Supervised fine-tuning (training/train_qlora.py) . . . 31
A.4 Base vs fine-tuned from one model (utils/model_loader.py) . . . 31
Appendix B — Glossary . . . 31

1. Executive Summary

EduTune AI is an AI teaching assistant for **Artificial Intelligence, Machine Learning, Deep Learning and Generative AI**. It was built by taking a general-purpose, pre-trained open-source language model and **specialising** it through **instruction tuning** — supervised fine-tuning on examples of the form *instruction → ideal teaching response*.

Fine-tuning a whole language model normally needs data-centre GPUs. This project instead uses **PEFT** (parameter-efficient fine-tuning) with **LoRA** adapters, and loads the frozen base model in **4-bit precision (QLoRA)**. As a result, the entire training ran on an ordinary 8 GB laptop GPU in about 17 minutes, updating only 1.18% of the parameters.

The result is used through a Streamlit web application that offers a tutor chat with nine teaching modes, a side-by-side **Base Model vs Fine-Tuned Model** comparison, a scored quiz, a viva examiner, an interview coach and an MCQ generator.

1.1 Key numbers

Item	Value
Base model	Qwen/Qwen2.5-1.5B-Instruct (1,543,714,304 parameters, frozen)
Trainable parameters (LoRA)	18,464,768 = 1.18% of the total
Dataset	840 examples: 756 training, 84 validation
Training	3 epochs, 285 optimiser steps, 16.5 minutes, peak GPU memory 2.81 GB
Output	LoRA adapter of 46 MB (the full model is ~3 GB)
Validation loss (response tokens)	2.1532 (base) → 0.9092 (fine-tuned)
Validation perplexity	8.612 (base) → 2.482 (fine-tuned): −71%
Tests	8 / 8 automated system tests pass

> **What to say in the presentation**
>
> “I took a pretrained language model and specialised it into an AI Teaching Assistant using instruction tuning. I used PEFT with LoRA, and QLoRA to reduce GPU memory requirements. The model was trained on educational instruction-response examples and can now perform different teaching tasks such as simple explanations, analogies, viva preparation, interview preparation, MCQs, and quizzes.”

2. Problem Statement and Objectives

2.1 Problem statement

General-purpose large language models (LLMs) are trained to be broadly helpful. They are **not optimised for a particular teaching style or domain**. When a student asks one to “explain LSTM to a beginner” or to “act as a viva examiner”, the answer's structure, depth and tone vary from question to question. They also rarely follow a fixed output format that an application can rely on — for example, a multiple-choice question with a machine-readable answer key.

Specialising a model by **fine-tuning** solves this, but full fine-tuning is expensive: updating all 1.5 billion parameters with the Adam optimiser needs roughly 25 GB of GPU memory, far beyond a student laptop.

2.2 Objectives

1. Build an **instruction-tuning dataset** of high-quality educational examples across AI, ML, DL and GenAI.
2. Perform **genuine supervised fine-tuning** (not just prompt engineering) using **PEFT + LoRA**.
3. Use **QLoRA (4-bit quantisation)** so the training fits on a consumer laptop GPU.
4. Evaluate the fine-tuned model against the base model **honestly**: quantitative metrics plus side-by-side answers, with no invented 'accuracy'.
5. Deliver an interactive **teaching application**: teaching modes, quiz, viva, interview, MCQs.
6. Keep the code modular and configurable (one `config.py`) and handle errors gracefully.

2.3 Scope

The assistant covers 37 topics, including AI fundamentals, intelligent agents, search, knowledge representation, supervised and unsupervised learning, regression, classification, clustering, decision trees, random forests, SVM, KNN, gradient descent, over- and underfitting, neural networks, backpropagation, activation functions, vanishing gradients, CNN, RNN, LSTM, GRU, GAN, VAE, transformers, attention, LLMs, prompt engineering, fine-tuning, LoRA, QLoRA, PEFT, instruction tuning, RLHF and DPO.

> **Why not just use ChatGPT with a good prompt?**
>
> Prompt engineering changes only the *input*; the model's weights stay the same, so its behaviour stays brittle and long prompts cost tokens on every request. The project requirement was to **actually change the model** — to teach it a behaviour through training — and to run it **locally** without any paid API. Fine-tuning gives consistent behaviour; LoRA and QLoRA make it affordable.

3. Background Concepts

This chapter explains the theory behind every technique used. These are the concepts examiners most often ask about.

3.1 Large language models and next-token prediction

An LLM is a transformer neural network trained on huge amounts of text to predict the **next token** (a word or piece of a word). Given “The cat sat on the”, it outputs a probability for every token in its vocabulary, and “mat” gets a high probability. Text is generated one token at a time: predict, append, repeat. Doing this over trillions of tokens teaches grammar, facts and reasoning patterns.

The training loss is **cross-entropy**: −log P(correct next token). A lower loss means the model assigns higher probability to the correct text.

3.2 Pre-training vs fine-tuning

	Pre-training	Fine-tuning
Data	Trillions of tokens of raw text	Hundreds to thousands of task examples
Goal	General language knowledge	Specialise for a task, domain or style
Cost	Millions of dollars, GPU clusters	Minutes to hours; here 16.5 min on a laptop
Who did it here	Alibaba (Qwen team), before this project	This project

Fine-tuning is a form of **transfer learning**: we reuse what the model already knows and only teach the new behaviour.

3.3 Instruction tuning and supervised fine-tuning (SFT)

Supervised fine-tuning means training on examples that contain the *desired output*. **Instruction tuning** is SFT where every example is an *instruction* (optionally with an *input*) paired with the ideal *response*. The model learns **how to respond** — style, structure and format — more than new facts.

```
{"instruction": "Explain LSTM using a simple real-world analogy.",
 "input": "",
 "output": "**Analogy:** Imagine you are reading a long story...",
 "category": "analogy"}
```

The optional **input** field carries extra context. In this project it holds, for example, a student's viva answer that the model must evaluate, or an explanation it must simplify.

3.4 PEFT — parameter-efficient fine-tuning

In **full fine-tuning** every weight is updated. That needs memory for the weights, their gradients and the optimiser state (Adam keeps two extra values per parameter), about 16 bytes per parameter. It also produces a complete new copy of the model for every task.

PEFT freezes the pre-trained model and trains only a **small number of extra parameters**. Gradients and optimiser states are needed only for those, so memory drops sharply, the saved result is small, and the original model is untouched. Methods include LoRA, adapters, prefix tuning, prompt tuning and IA3. This project uses **LoRA** through the Hugging Face `peft` library.

3.5 LoRA — low-rank adaptation

For a frozen weight matrix **W** of size d × k, LoRA does not learn a full update ΔW (d × k numbers). It learns two thin matrices, **B** (d × r) and **A** (r × k), whose product is the update: ΔW = B·A. The rank **r** is small (16 here). The layer then computes:

LoRA forward pass

`h = W·x + (α / r) · B·A·x`

W is frozen; only A and B are trained. α (lora_alpha = 32) scales the update. B starts at zero, so at step 0 the model behaves exactly like the original.

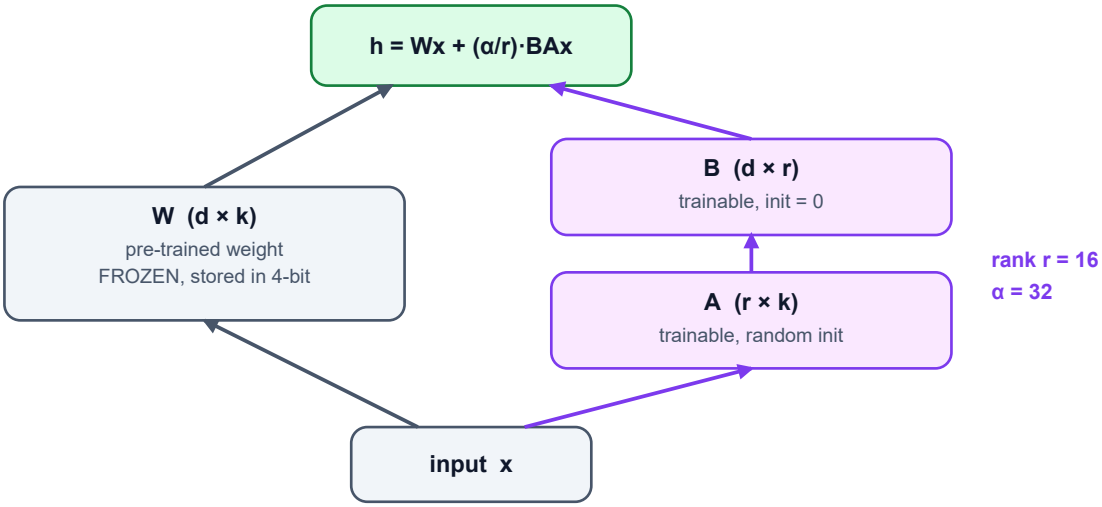


Figure 1 — LoRA inside one linear layer

Worked example (from this model). A 1536 × 1536 attention projection has 2,359,296 weights. With r = 16, LoRA trains A (16 × 1536) and B (1536 × 16) = 16 × (1536 + 1536) = **49,152** parameters, which is 2.1% of that layer.

Why it works: research found that the weight change needed to adapt a pre-trained model has a low "intrinsic rank", so a low-rank update captures most of it.

3.6 Quantisation and QLoRA

Quantisation stores numbers with fewer bits. A 16-bit weight uses 2 bytes; a 4-bit weight uses 0.5 bytes. QLoRA (Dettmers et al., 2023) combines a **4-bit quantised, frozen base model** with **LoRA adapters kept in 16-bit (bf16)**. It introduced three ideas:

- **NF4 (4-bit NormalFloat)** — a 4-bit data type whose 16 levels are placed to suit normally distributed neural-network weights.
- **Double quantisation** — the scaling constants of each weight block are themselves quantised, saving about 0.4 bits per parameter.
- **Paged optimisers** — optimiser memory can spill to CPU RAM during spikes instead of crashing with "out of memory".

During each forward and backward pass, the 4-bit weights are **dequantised on the fly** to bf16 for the matrix multiplication. Gradients flow *through* the frozen weights into the LoRA matrices. The base weights never change, so rounding error in them does not accumulate, and the adapters can learn to compensate for it.

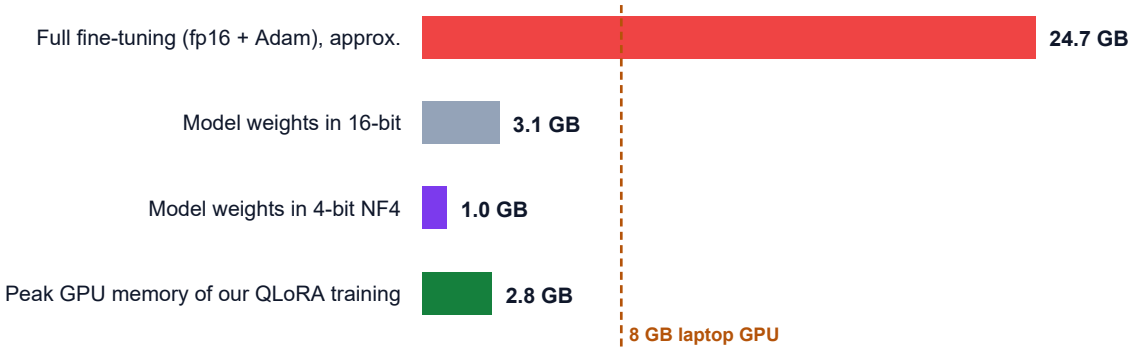


Figure 2 — GPU memory: why QLoRA makes training fit on a laptop

3.7 Completion-only loss

Each training text contains a system message, the student's instruction and the response. We want the model to learn to **produce responses**, not to reproduce questions. So the loss is computed only on the response tokens; the prompt tokens are **masked** (label = −100). In TRL this is `completion_only_loss=True` with a prompt/completion dataset.

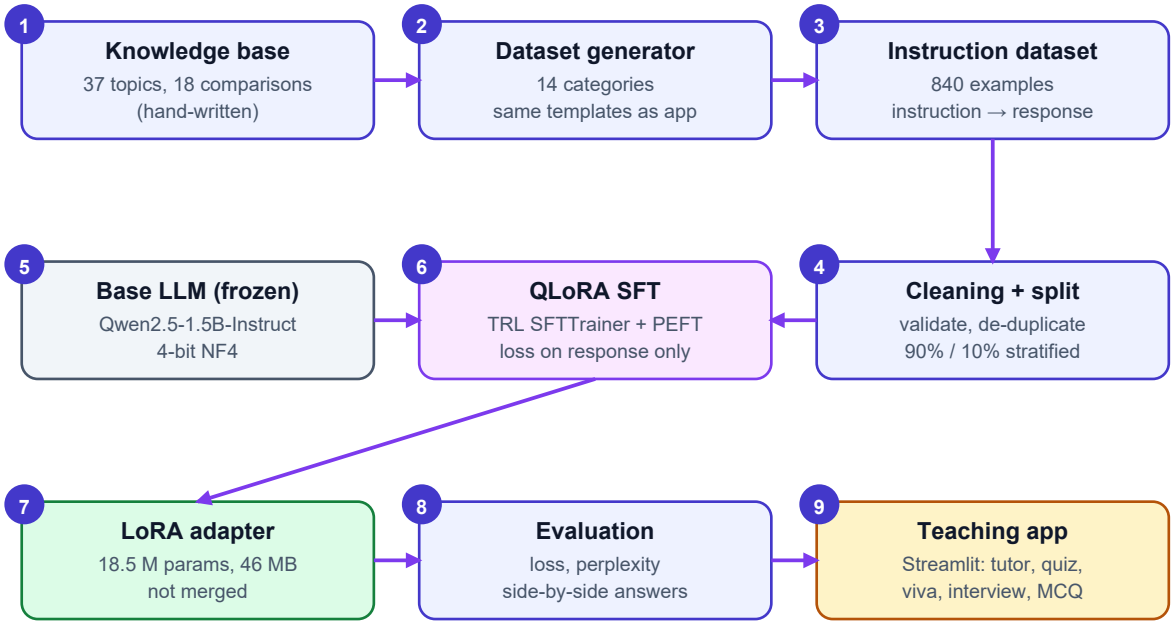
3.8 Perplexity — and why not “accuracy”

Perplexity = e^{loss}. It can be read as “how many tokens the model is choosing between” at each step: perplexity 8.6 means the model is roughly as uncertain as picking among 8.6 equally likely tokens; 2.5 means it is much more confident about the reference answer.

Classification accuracy is **not meaningful** for free-form explanations: many different answers can be correct, and exact matching would count them all as wrong. So the project uses validation loss and perplexity for measurement, and side-by-side reading for quality — and it never claims one automatic “quality score”.

4. System Architecture

The project is a pipeline of nine stages. Each stage is a separate script or module, so it can be run, tested and explained on its own.



Data preparation (1-4) · Training (5-7) · Evaluation & use (8-9)

Figure 3 — End-to-end pipeline

#	Stage	File	What happens
1	Knowledge base	scripts/knowledge/*.py	Hand-written, structured content for 37 topics and 18 comparisons
2	Dataset generator	scripts/generate_dataset.py	Turns every field into instruction→response pairs in 14 categories
3	Instruction dataset	data/instruction_dataset.jsonl	840 JSON records
4	Cleaning + split	scripts/prepare_dataset.py	Validates, removes empty/duplicate records, 90/10 stratified split
5	Base LLM	utils/model_loader.py	Loads Qwen2.5-1.5B-Instruct in 4-bit NF4, frozen
6	QLoRA SFT	training/train_qlora.py	Adds LoRA adapters and trains them with TRL's SFTTrainer
7	LoRA adapter	models/edu_tune_lora/	Saved separately — not merged into the base model
8	Evaluation	evaluation/evaluate.py	Loss and perplexity for both models + side-by-side answers
9	Teaching app	app.py	Streamlit interface using the base model + adapter

4.1 Inference architecture: one model, two behaviours

At run time, the base weights are loaded **once** and the adapter is attached on top. For a “Base Model” answer the adapter is temporarily switched off with PEFT's `disable_adapter()`. This halves GPU memory compared with loading two models, and it guarantees that the **only** difference between the two answers is the trained adapter — the same weights, the same prompt and the same system message.

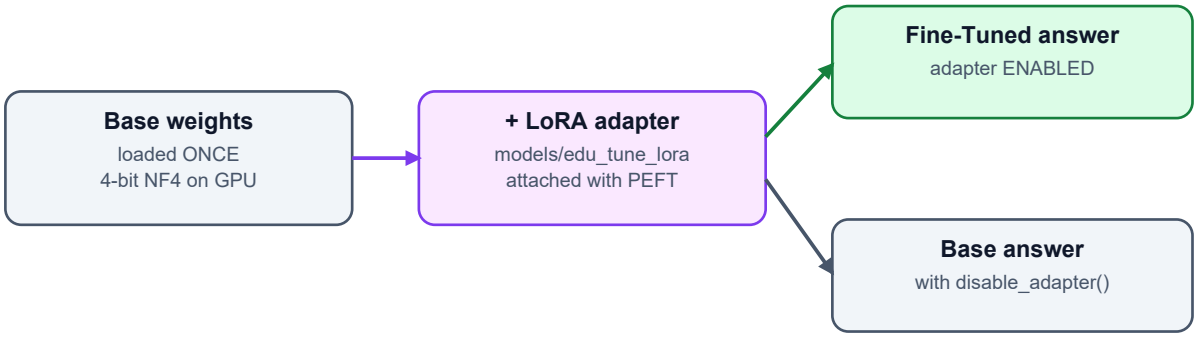


Figure 4 — How the app produces base and fine-tuned answers from one loaded model

5. Technology Stack

Technology	Version	Role in this project
Python	3.11	Language for the whole pipeline
PyTorch	2.11 (CUDA 12.8)	Tensors and GPU execution; the cu128 build is required for RTX 50-series (Blackwell) GPUs
Transformers	5.17	Loads the model and tokenizer, chat template, text generation, BitsAndBytesConfig
Datasets	5.0	Loads the JSONL training/validation files for the trainer
PEFT	0.21	LoraConfig, get_peft_model (adds adapters, freezes base), PeftModel, disable_adapter
TRL	1.14	SFTTrainer / SFTConfig: supervised fine-tuning with completion-only loss
bitsandbytes	0.50	4-bit NF4 quantisation and the paged 8-bit AdamW optimiser
Accelerate	1.15	Device placement and mixed precision inside the Trainer
Streamlit	1.64	Web interface: chat, comparison, quiz, viva, interview, MCQ pages
Pandas / NumPy	3.0 / 2.4	Training history, statistics, tables
scikit-learn	1.9	Stratified train/validation split
Matplotlib	3.11	Training-loss plot

No paid API is used — everything runs locally. The model is downloaded once from Hugging Face (Apache-2.0 licence).

6. Project Structure

Path	Purpose
config.py	ALL settings: model name, paths, hyper-parameters, LoRA, 4-bit, generation, disclaimer
app.py	Streamlit application (6 pages)
utils/device.py	CUDA detection, precision choice (bf16/fp16/fp32), out-of-memory help text
utils/model_loader.py	load_base_model(), load_finetuned_model(), TeachingModel (adapter on/off, streaming)

Path	Purpose
utils/prompts.py	Teaching modes, quick actions, build_prompt(), viva/interview/MCQ prompt builders
utils/mcq.py	Parses MCQ text into questions/options/answers; curated question bank
training/format_dataset.py	format_instruction() — the single place where records become model input
training/train_qlora.py	QLoRA training, parameter report, logging, adapter saving
evaluation/evaluate.py	Validation loss/perplexity + side-by-side generation
evaluation/comparison.py	Neutral, measurable response metrics (no 'winner')
scripts/knowledge/	Structured educational content: ai, ml, dl, genai topics + comparisons
scripts/generate_dataset.py	Builds the instruction dataset
scripts/prepare_dataset.py	Cleaning, validation, statistics, split
scripts/test_model.py	PASS/FAIL system tests
data/	instruction_dataset.jsonl, train.jsonl, validation.jsonl, raw/knowledge_base.json
models/edu_tune_lora/	The trained LoRA adapter (adapter_config.json, adapter_model.safetensors)
results/	training_history.csv, training_loss.png, training_summary.json, eval_metrics.json, comparison.json

7. The Dataset

7.1 Design approach

Quality matters more than quantity in instruction tuning, and the brief warned against repetitive filler. So the content was written **once, by hand, as structured knowledge** for each topic, and a generator turns it into many different teaching tasks. Every fact is written once, which keeps the answers consistent, and each task type (definition, analogy, viva feedback, MCQ...) gets a distinct, well-formatted response.

Domain	Topics
Artificial Intelligence	Artificial Intelligence (AI), Intelligent Agents, Search Algorithms in AI, Knowledge Representation
Machine Learning	Supervised Learning, Unsupervised Learning, Regression (Linear Regression), Classification, Clustering (K-Means), Decision Trees, Random Forest, Support Vector Machine (SVM), K-Nearest Neighbours (KNN), Gradient Descent, Overfitting, Underfitting
Deep Learning	Artificial Neural Network (ANN), Backpropagation, Activation Functions, Vanishing and Exploding Gradients, Convolutional Neural Network (CNN), Recurrent Neural Network (RNN), Long Short-Term Memory (LSTM), Gated Recurrent Unit (GRU)
Generative AI	Generative Adversarial Network (GAN), Variational Autoencoder (VAE), Transformer Architecture, Attention Mechanism, Large Language Model (LLM), Prompt Engineering, Fine-Tuning, LoRA (Low-Rank Adaptation), QLoRA (Quantized LoRA), PEFT (Parameter-Efficient Fine-Tuning), Instruction Tuning, RLHF (Reinforcement Learning from Human Feedback), DPO (Direct Preference Optimization)

7.2 Fields written for every topic

Field	Used to create
definition, question	Definition answers; exam-ready lines
simple, eli10	Beginner and “explain like I'm 10” explanations; simplification targets
analogy	Real-world analogy answers
how_it_works (steps)	Detailed, step-by-step technical explanations
example	Worked examples
applications, advantages, limitations	Applications and pros/cons answers
key_points	Revision notes, “key idea to remember”
viva (Q&A; pairs)	Viva question lists, single examiner questions, feedback on student answers
interview (Q&A; pairs)	Interview answers, interviewer questions, evaluation of candidate answers
mcqs	Multiple-choice question sets and single quiz questions
numerical (optional)	Step-by-step numerical problems (e.g. CNN output size, LoRA parameter count)
troubleshooting (optional)	Diagnose-and-fix answers (e.g. NaN loss, CUDA out of memory)

7.3 Generated categories

The generator produced **14 categories**. The instructions are phrased with the **same templates the app uses** (`utils/prompts.py`), so the model is trained on exactly the prompts it sees at demo time. Wording is varied with a seeded random generator, so the model learns the task rather than one phrasing.

Category	Count	Example instruction
interview	179	Ask me one technical interview question about instruction tuning. Only ask the question.
viva	148	Ask me one viva question about fine-tuning. Only ask the question.
simple_explanation	111	Explain like I'm 10 years old: What is classification in machine learning?
mcq	74	Create 2 multiple-choice questions on RLHF.
numerical	40	Give a numerical problem with a step-by-step solution on: neural networks
example	37	Give an example of decision trees.
analogy	37	Give an analogy for KNN.
definition	37	Define unsupervised learning.
application	37	What are the applications of QLoRA?
advantages_disadvantages	37	What are the advantages and disadvantages of underfitting?
revision	37	Give quick revision notes on: supervised learning
detailed_explanation	37	Give a detailed, well-structured explanation: What is backpropagation?
comparison	18	What is the difference between LSTM and GRU?
troubleshooting	11	My training accuracy is 99% but my validation accuracy is only 70%. How do I fix it?

7.4 Example of a record that uses the input field

Viva feedback teaches the model to act as an examiner. The student's answer is in **input**:

```
instruction: You are a viva examiner. Evaluate the student's answer to the viva question: "How is
    underfitting different from overfitting?" Give educational feedback.

input: Student's answer: Underfitting means the model performs poorly even on training data (high bias);
    overfitting means it performs well on training data but poorly on new data (high variance).

output:
**Verdict:** Correct.

**What you got right:** Your answer states the key idea accurately and concisely.

**To impress the examiner:** Add a concrete example. Predicting salary from years of experience when
    salary actually grows quickly at first and then levels off.

**Model answer:** Underfitting means the model performs poorly even on training data (high bias);
    overfitting means it performs well on training data but poorly on new data (high variance).
```

7.5 Cleaning and splitting

`scripts/prepare_dataset.py` performs these steps:

1. Load the JSONL file; malformed lines are reported, not fatal.
2. Remove invalid records (missing fields or wrong types).
3. Remove empty instructions and empty outputs.
4. Remove duplicates, comparing text case- and whitespace-insensitively.
5. Validate the required fields (instruction, input, output).
6. Split 90% / 10%, **stratified by category**, so every category appears in validation.

Statistic	Value
Total examples	840
Training / validation	756 / 84
Records removed by cleaning	0 (the generator already produces valid, unique records)
Domains	Generative AI: 283, Machine Learning: 268, Deep Learning: 185, Artificial Intelligence: 86, Mixed: 18
Response length (words)	min 3, mean 77, max 314

7.6 Instruction formatting

`training/format_dataset.py` holds the single `format_instruction()` function. Each record becomes a conversation — system message, user message (instruction + input), assistant message (output) — rendered with the model's **own chat template**. This is exactly what the model sees:

```
<|im_start|>system
You are EduTune AI, a patient and accurate teaching assistant for Artificial Intelligence, Machine
    Learning, Deep Learning and Generative AI.<|im_end|>
<|im_start|>user
Explain LSTM using a simple real-world analogy.<|im_end|>
<|im_start|>assistant
**Analogy:** Imagine you are reading a long story...<|im_end|>
```

A readable “### Instruction / ### Response” (Alpaca-style) version is also available for display and for models without a chat template. The same system message is used in training and in the app, so the **only** difference between base and fine-tuned answers is the adapter.

8. Model and Training

8.1 Choice of base model

Qwen/Qwen2.5-1.5B-Instruct — 1.54 billion parameters, 28 transformer layers, hidden size 1536, Apache-2.0 licence. It is small enough to fine-tune with QLoRA on an 8 GB laptop GPU, it is already instruction-capable (it has a chat template), and it is strong for its size. The model is set in one place — `MODEL_NAME` in `config.py` — so it can be swapped for, e.g., Qwen2.5-0.5B (for weaker hardware) or Qwen2.5-3B.

8.2 Training configuration

Setting	Value	Why
Quantisation	4-bit NF4 + double quant., bf16 compute	QLoRA: fits the 8 GB GPU
LoRA rank r / alpha / dropout	16 / 32 / 0.05	Standard starting point; alpha = 2r
Target modules	q, k, v, o, gate, up, down projections	Adapting attention + MLP layers gives the best quality
Epochs	3	Enough for the loss to plateau
Batch size × grad. accumulation	2 × 4 = 8	Small micro-batches save memory; accumulation keeps the effective batch at 8
Learning rate / schedule	0.0002 / cosine with 5% warm-up	Typical LoRA LR (higher than full fine-tuning)
Max sequence length	1024	Longest examples fit without truncation
Optimiser	paged AdamW 8-bit	QLoRA's memory-safe optimiser
Gradient checkpointing	On	Recomputes activations to save memory
Loss	Completion-only cross-entropy	Learn answers, not questions

8.3 Where the trainable parameters come from

LoRA adds A and B to each of the 7 projection layers in all 28 decoder layers. The count below exactly matches what the training script printed:

Module	Shape (in → out)	Full weights	LoRA params r·(in+out)
q_proj	1536 → 1536	2,359,296	49,152
k_proj	1536 → 256	393,216	28,672
v_proj	1536 → 256	393,216	28,672
o_proj	1536 → 1536	2,359,296	49,152
gate_proj	1536 → 8960	13,762,560	167,936
up_proj	1536 → 8960	13,762,560	167,936
down_proj	8960 → 1536	13,762,560	167,936
Per decoder layer			**659,456**
× 28 layers			**18,464,768**

Printed by training/train_qlora.py

```
Total parameters: 1,562,179,072
Trainable parameters: 18,464,768
Trainable percentage: 1.1820%
```

8.4 What the training script does, step by step

1. Prints the hardware (CUDA available, GPU name, memory) — and warns clearly if there is no GPU.
2. Loads train/validation JSONL and converts each record to a prompt/completion conversation.
3. Loads the base model in 4-bit (`BitsAndBytesConfig`) and prepares it for k-bit training.
4. Wraps it with `get_peft_model(model, LoraConfig(...))`: base frozen, adapters trainable.
5. Prints total / trainable parameters and the trainable percentage (the PEFT demonstration).
6. Trains with TRL's `SFTTrainer`, logging epoch, step, training loss, validation loss and learning rate.
7. Catches CUDA out-of-memory errors and prints which config values to reduce.
8. Saves **only the adapter** to `models/edu_tune_lora/` (not merged), plus the history CSV, loss plot and a summary JSON.

8.5 Training results

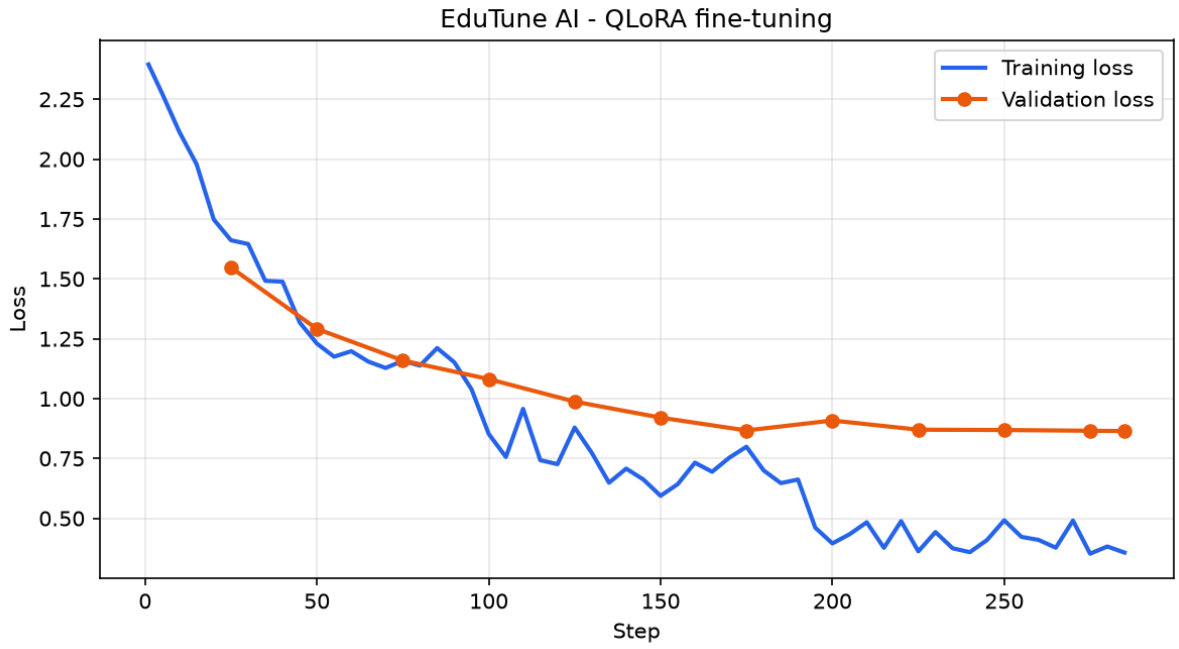


Figure 5 — Training and validation loss (results/training_loss.png)

How to read the curve (important for the viva)

Validation loss falls quickly in epoch 1 (1.55 → 1.08), keeps improving in epoch 2, and **plateaus from about step 175** (0.867 → 0.864). Meanwhile, training loss keeps falling to about 0.4. This widening gap in epoch 3 is the **beginning of memorisation**: the model fits the training examples more closely without generalising better. Validation loss does not rise, so the model is not harmed, but **2 epochs would have given almost the same result**. Early stopping on validation loss would be the principled fix.

Step	Epoch	Validation loss	Perplexity
25	0.265	1.5475	4.70
50	0.529	1.2911	3.64
75	0.794	1.1592	3.19
100	1.053	1.0814	2.95
125	1.317	0.9875	2.68
150	1.582	0.9206	2.51
175	1.847	0.8670	2.38
200	2.106	0.9085	2.48
225	2.37	0.8701	2.39
250	2.635	0.8688	2.38
275	2.899	0.8657	2.38
285	3.0	0.8645	2.37

Result	Value
Runtime	16.52 minutes
Peak GPU memory	2.81 GB (of 8 GB)
Final validation loss / perplexity (trainer)	0.8645 / 2.374
Average training loss over the run	0.8618
Adapter size	46.17 MB

9. Evaluation

9.1 Methodology

`evaluation/evaluate.py` compares the **same loaded model** with the adapter off (base) and on (fine-tuned):

- **Quantitative:** validation loss and perplexity on the 84 held-out examples, counting **only the response tokens**, with a per-category breakdown.
- **Qualitative:** both models answer 14 fixed questions with greedy (deterministic) decoding. These include the required ones (Explain RNN, Explain LSTM, vanishing gradient, Compare RNN and LSTM, CNN for beginners, overfitting, LoRA, QLoRA), several teaching modes, and **3 topics that are not in the training data** (batch normalization, dropout, word embeddings).
- For each pair, **measurable differences** are recorded — length, structure, analogy/example use, MCQ format compliance. **No automatic winner is declared.**

9.2 Quantitative results

Metric	Base model	Fine-tuned model	Change
Validation loss (response tokens)	2.1532	0.9092	−58%
Perplexity	8.612	2.482	−71%
Response tokens scored	8,911	8,911	

Note: the evaluation script scores every response token equally, so its fine-tuned loss (0.9092) differs slightly from the trainer's own figure (0.8645), which averages per batch.

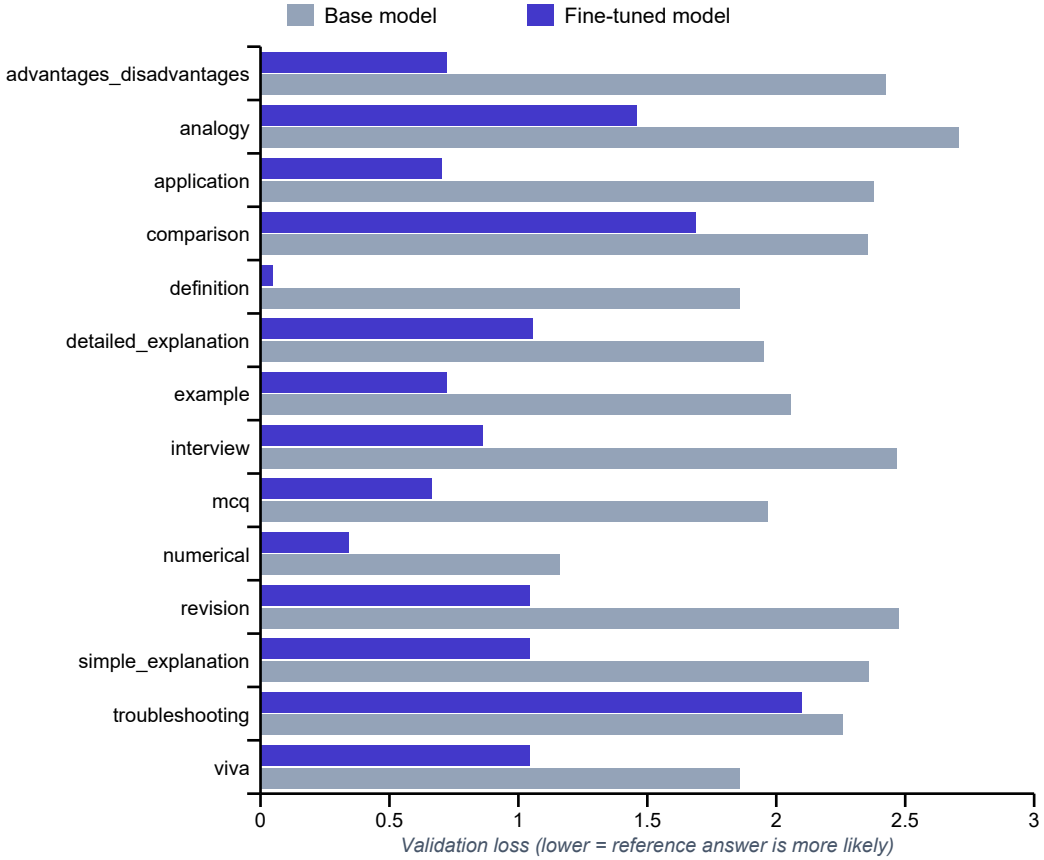


Figure 6 — Validation loss per category, base vs fine-tuned

Interpreting the per-category results honestly

- The loss drops in **every** category, most for structured formats (definitions, MCQs, numerical, application).
- **Definition** falls to 0.0473. This is suspiciously low, and the reason is instructive: the same definition text also appears inside other training examples for that topic (e.g. analogy and detailed answers). The model has effectively seen it — a form of **overlap/leakage** between train and validation.
- **Troubleshooting** barely improves (2.2583 → 2.098), but there is only 1 validation example, so this is not statistically meaningful.
- Loss measures how likely the model finds the *reference* answer — the learned style and content — not whether an answer is factually correct.

9.3 Qualitative comparison (real outputs from results/comparison.json)

Prompt: “Explain like I'm 10 years old: What is an RNN?”

BASE MODEL	FINE-TUNED MODEL
An RNN (Recurrent Neural Network) is a type of neural network that can remember information from the past. It's called "recurrent" because it can look back at what happened before in order to make better predictions or decisions. Imagine you're playing a game where you have to guess what your friend will do next based on their previous moves - an RNN would be really good at this! It remembers all the moves they've made so far and uses them to help predict what they'll do next. This makes RNNs very useful for tasks that involve remembering things over time, like language translation or speech recognition.	Imagine reading a story one word at a time. Each new word helps you guess the next one. An RNN keeps remembering what it has read so far when it predicts the next word. **The grown-up version:** A Recurrent Neural Network is a neural network designed for sequential data.

- Response length: base 105 words, fine-tuned 50 words (52% shorter).
- Response structure: base uses plain paragraphs; fine-tuned uses 1 bold labels.
- Sentence complexity: 21.0 vs 12.5 words per sentence (lower is usually easier to read).
- Teaching style: both responses use an analogy.
- Simplicity for the audience: average word length 4.71 vs 4.26 characters.

Prompt: “Create 2 multiple-choice questions on RNN.”

BASE MODEL	FINE-TUNED MODEL
Sure! Here are two multiple-choice questions related to Recurrent Neural Networks (RNNs): 1. What is the primary advantage of using an LSTM over a standard RNN? A) Faster training speed B) Ability to handle long-term dependencies C) Increased computational efficiency D) Reduced need for gradient descent optimization 2. In which scenario would you most likely use a GRU instead of an LSTM in an RNN model? A) When dealing with very large datasets B) For tasks requiring short-term memory C) To improve the vanishing gradient problem D) To increase the number of hidden units in the network Please note that these questions are designed based on common practices and theoretical advantages associated with different types of RNN architectures. The correct answers can vary depending on specific application contexts.	**Question 1:** Which loss function is commonly used with vanilla RNNs? A. Cross-entropy B. Mean Squared Error C. Binary cross-entropy D. Categorical cross-entropy **Answer:** A **Explanation:** Vanilla RNNs use the softmax output and categorical cross-entropy loss. **Question 2:** What does vanishing gradients mean? A. The model's weights become zero B. Gradients shrink as they are backpropagated through many layers or time steps C. All activations become negative D. Too much training data **Answer:** B **Explanation:** Repeated multiplication of small derivatives causes early-layer gradients to vanish.

- Response length: base 131 words, fine-tuned 90 words (31% shorter).
- Response structure: base uses 2 numbered steps; fine-tuned uses 6 bold labels.
- Sentence complexity: 18.7 vs 7.5 words per sentence (lower is usually easier to read).
- Instruction following (MCQ format): 0 vs 2 questions parsed in the expected A-D + Answer + Explanation format.
- Instruction following: 2 items were requested.

Prompt: “Answer this viva question clearly and concisely, as a student would to an examiner: What is dropout?”

BASE MODEL	FINE-TUNED MODEL
Dropout is a regularization technique used in neural networks to prevent overfitting. It involves randomly "dropping out" or setting the value of some neurons' activations to zero during training. This process helps to reduce the complexity of the model by preventing it from learning too many parameters that do not generalize well to unseen data. Dropout is typically applied during the hidden layers of a neural network, where it can help improve the generalization ability of the model on new data.	Dropout randomly sets a certain fraction of neurons' outputs to zero at each update, which acts as a form of regularisation and improves generalisation. **Exam-ready definition:** Dropout randomly drops units (neurons or connections) during training, so each layer sees a random subset of activations.

- Response length: base 81 words, fine-tuned 45 words (44% shorter).
- Response structure: base uses plain paragraphs; fine-tuned uses 1 bold labels.
- Sentence complexity: 20.2 vs 22.5 words per sentence (lower is usually easier to read).

9.4 Findings

- **Format following improved clearly.** Asked for 2 MCQs, the base model wrote questions with *no answer key* (0/2 parseable); the fine-tuned model produced 2/2 in the exact Question / A–D / Answer / Explanation format that the Quiz page needs.
- **Consistent teaching structure.** Fine-tuned answers are shorter and use labels learned in training (“Key idea to remember”, “The grown-up version”, “Exam-ready definition”).
- **Style transfers to unseen topics.** For dropout — which was not in the dataset — the model still answered in the learned viva style.
- **Not always better.** For “Explain CNN to a beginner” the base model's longer answer included an example and an analogy that the short fine-tuned answer did not. Fine-tuned MCQs can contain errors: in the example above, options A and D overlap, and in testing one quiz question wrongly described an “RL loop” in DPO. Instruction tuning on ~750 examples mainly changes behaviour, not the 1.5B model's knowledge.

What to say in the presentation

“Perplexity on held-out teaching answers dropped by 71%, and the fine-tuned model reliably follows the formats it was trained on — for example, 2 out of 2 parseable MCQs versus 0 for the base model. But I don't claim it is better at everything: it is shorter and can still make factual mistakes, which is why the app shows a disclaimer.”

10. The Teaching Application

Run with `streamlit run app.py` and open http://localhost:8501. The model and adapter load once (about 15 seconds) and are cached with `st.cache_resource`. Answers are streamed token by token.

10.1 Sidebar

- **Navigation** between the six pages.
- **Teaching mode** — changes the instruction sent to the model (see 10.3).
- **Topic** — used by quick actions, quiz, viva, interview and MCQ pages (or “Any”).
- **Generation settings** — max new tokens and temperature.
- The **safety disclaimer**: “EduTune AI is an educational assistant. Its answers may contain mistakes. Verify important academic information with textbooks, lecturers, or authoritative sources.”

10.2 Pages

Page	What it does
AI Tutor	Chat with the fine-tuned tutor in the chosen teaching mode, plus seven quick-action buttons. Shows the exact instruction sent to the model.
Base vs Fine-Tuned	The same question to both models, side by side, with deterministic decoding and a list of observable differences. Also browses the saved evaluation results.
Quiz Mode	One MCQ at a time, A–D options, “Correct!” or the right answer with an explanation, running score (e.g. Score: 4 / 5). Questions come from the model (parsed) or from the curated bank.
Viva Mode	The AI examiner asks one question; the student answers; the examiner gives a verdict, what was right, what to add, and a model answer; then the next question.
Interview Mode	Interview question → candidate answer → What you got right / What you missed / Suggested improved answer.
MCQ Generator	Topic, number of questions (1–10), Easy/Medium/Hard → MCQs with hidden answers and explanations.

10.3 Teaching modes

Mode	Instruction template sent to the model
Simple Explanation	Explain in simple, beginner-friendly language: {question}
Detailed Explanation	Give a detailed, well-structured explanation: {question}
Explain Like I'm 10	Explain like I'm 10 years old: {question}
Example-Based Learning	Teach this using a concrete worked example: {question}
Interview Preparation	Answer this as a strong candidate would in a technical interview: {question}
Viva Preparation	Answer this viva question clearly and concisely, as a student would to an examiner: {question}
MCQ Practice	Create 3 multiple-choice questions to test my understanding of: {question}
Numerical Problem	Give a numerical problem with a step-by-step solution on: {question}
Revision	Give quick revision notes on: {question}

10.4 Quick actions

Button	Instruction
Explain	Explain {topic}.
Give Example	Give a real-world example of {topic}.
Give Analogy	Explain {topic} using a simple real-world analogy.
Quiz Me	Create 3 multiple-choice questions on {topic}.
Viva Questions	Give 5 viva questions on {topic} with short answers.
Interview Questions	Give interview questions on {topic} with model answers.
Simplify Answer	Simplify this explanation so that a beginner can understand it. (the previous answer is passed as input)

10.5 How Quiz, Viva and Interview use the fine-tuning

These pages depend on formats the model **learned during training**. The dataset contains “Create 1 multiple-choice question on X” examples in a fixed format that `utils/mcq.py` can parse, “Ask me one viva question about X” examples, and examiner-feedback examples whose student answer is in the input field. If the model's MCQ output cannot be parsed, the quiz falls back to the curated question bank and tells the user so.

10.6 Error handling

Situation	What the user sees
Model cannot be loaded (no internet, wrong name)	A clear message and a “Retry loading model” button — no traceback
LoRA adapter missing	A warning explaining how to train it; the base model still works on the comparison page
CUDA not available	Printed warning in the scripts; the app runs on CPU (slowly)
CUDA out of memory	A message listing exactly which config.py values to reduce
Missing or invalid dataset	Scripts stop with instructions to run the generator/preparation steps
Empty question or answer	“Please type a question first.”
Generation error	A friendly error message; the app keeps running

11. Hardware and Performance

Aspect	This project
GPU	NVIDIA GeForce RTX 5050 Laptop GPU, 8 GB, Blackwell architecture (needs PyTorch built for CUDA 12.8)
Precision	Weights 4-bit NF4; computation and adapters in bfloat16
Training memory	Peak 2.81 GB
Training time	16.52 minutes for 3 epochs
Inference	One model in memory; answers are streamed
CPU fallback	Detected automatically with a clear warning; 4-bit training needs a GPU

If memory is insufficient, all relevant settings are in `config.py`: BATCH_SIZE = 1 (raise GRADIENT_ACCUMULATION_STEPS), MAX_SEQ_LENGTH = 512, keep GRADIENT_CHECKPOINTING and USE_4BIT on, LORA_R = 8, or a smaller MODEL_NAME such as Qwen2.5-0.5B-Instruct.

12. Testing

`python scripts/test_model.py` runs end-to-end checks with clear PASS/FAIL output:

```
[PASS] GPU detection              CUDA available - NVIDIA GeForce RTX 5050 Laptop GPU (7.96 GB)
[PASS] Dataset loading            instruction_dataset=840, train=756, validation=84
[PASS] Dataset formatting         Alpaca + chat prompt/completion formats OK
[PASS] Prompt building            9 teaching modes, empty question rejected
[PASS] MCQ parser                 MCQ format parsed
[PASS] Base model loading         Qwen/Qwen2.5-1.5B-Instruct loaded (4-bit) on cuda:0
[PASS] Fine-tuned model loading   LoRA adapter attached (18,464,768 adapter parameters)
[PASS] Inference                  base: 'Sure! A Recurrent Neural Network (RNN) is a type o'
                                  fine-tuned: 'An RNN is like reading a long story one word at a '

8/8 tests passed
```

The Streamlit app was also exercised headlessly with Streamlit's AppTest: every page rendered without exceptions, and chat, quick actions, quiz scoring and viva feedback worked end to end.

13. How to Install and Run

```
# 1. Virtual environment (Windows)
py -3.11 -m venv venv
venv\Scripts\activate

# 2. PyTorch with CUDA, then the rest
pip install torch==2.11.0 --index-url https://download.pytorch.org/whl/cu128
pip install -r requirements.txt

# 3. Pipeline
python utils/device.py                # check the GPU
python scripts/generate_dataset.py    # build the dataset
python scripts/prepare_dataset.py     # clean + split
python training/train_qlora.py        # QLoRA training (~17 min)
python evaluation/evaluate.py         # base vs fine-tuned evaluation
python scripts/test_model.py          # PASS/FAIL tests

# 4. Application
streamlit run app.py                  # http://localhost:8501
```

Inference from Python, without the UI:

```
from utils.model_loader import TeachingModel
from utils.prompts import build_prompt

tm = TeachingModel()                                  # base + LoRA adapter, loaded once
q = build_prompt("What is an LSTM?", "Explain Like I'm 10")
print(tm.generate(q, use_adapter=True))               # fine-tuned
print(tm.generate(q, use_adapter=False))              # base model
```

14. Presentation Guide

14.1 Suggested slide outline (about 12–15 minutes)

#	Slide	Content	Time
1	Title	EduTune AI — instruction-tuned teaching assistant	0:30
2	Problem	General LLMs lack a consistent teaching style; full fine-tuning is too expensive	1:00
3	Objective	Specialise a small open model with instruction tuning on a laptop GPU	0:30
4	Architecture	Pipeline diagram (Figure 3)	1:00
5	Dataset	37 topics → 840 examples, 14 categories, example record	1:30
6	Concepts	Fine-tuning, instruction tuning, SFT, PEFT	1:00
7	LoRA + QLoRA	Figure 1, NF4, memory chart (Figure 2), 1.18% trainable	1:30
8	Training	Configuration + loss curve (Figure 5)	1:00
9	Evaluation	Perplexity −71%, per-category chart, honest findings	1:30
10	LIVE DEMO	See 14.2	4:00
11	Limitations & future work	Chapter 16	0:45
12	Conclusion	Summary sentence + thank you	0:30

14.2 Live demo script

Start the app a minute before presenting so the model is already loaded.

Step	Do this	Say this
1. Dataset	Open data/instruction_dataset.jsonl (or run prepare_dataset.py for the statistics)	“This is an instruction dataset: each record is an instruction and the ideal teaching response.”
2. Base model	Base vs Fine-Tuned page, question “Explain RNN to a beginner.” — look at the left column	“This is the pre-trained model with no training from me.”
3. QLoRA	Show config.py LoRA settings and results/training_summary.json	“The base model is quantised to 4-bit and frozen; only LoRA adapters — 1.18% of the parameters — are trained.”
4. Fine-tuned	Look at the right column for the same question	“Same weights, same prompt — the only difference is my 46 MB adapter.”
5. Differences	Scroll to “Observable differences”	“I report measurable differences instead of claiming an automatic score.”
6. ELI10	AI Tutor → mode “Explain Like I'm 10” → “What is an LSTM?”	“The mode changes the instruction; the model was trained on this exact phrasing.”
7. Viva	Viva Mode → Ask me a question → give a partial answer	“The model learned to act as an examiner from feedback examples in the dataset.”
8. MCQ / Quiz	Quiz Mode → answer 2–3 questions; MCQ Generator → 5 medium on RNN	“The fine-tuned model produces a parseable answer key; the base model doesn't.”

Step	Do this	Say this
9. Statistics	Show results/training_loss.png and eval_metrics.json	“Validation perplexity dropped from 8.6 to 2.5; training took 16.5 minutes on this laptop.”

Demo safety tips

- Use the **Base vs Fine-Tuned** page with deterministic decoding for the comparison — results are repeatable.
- For the quiz, the **Curated question bank** option is 100% reliable if a model-generated question looks wrong.
- Never re-run training during the demo — it overwrites the adapter.
- If the GPU is busy or the model fails to load, the saved results in the comparison page still work.

14.3 Numbers to remember

Question	Answer
Base model?	Qwen2.5-1.5B-Instruct, 1.54 B parameters, 28 layers
How many parameters trained?	18,464,768 — 1.18%
LoRA settings?	r = 16, alpha = 32, dropout 0.05, 7 projection modules
Quantisation?	4-bit NF4 with double quantisation, bf16 compute
Dataset?	840 examples, 37 topics, 14 categories, 756/84 split
Training?	3 epochs, 285 steps, LR 2e-4, effective batch 8, 16.5 min, 2.81 GB peak
Result?	Perplexity 8.612 → 2.482 (−71%)
Adapter size?	46 MB vs ~3 GB for the full model

15. Expected Viva Questions and Answers

Q1. What is the difference between fine-tuning and prompt engineering?

Prompt engineering changes only the input; the model's weights stay the same. Fine-tuning changes the weights by training on examples, which gives more consistent behaviour without long prompts. This project fine-tunes.

Q2. What is instruction tuning?

Supervised fine-tuning on (instruction, optional input, response) examples, so the model learns to follow requests in the desired style and format.

Q3. Why did you use LoRA instead of full fine-tuning?

Full fine-tuning of 1.5 B parameters needs ~25 GB (weights, gradients and Adam states). LoRA trains only 1.18% of the parameters, so gradients and optimiser states are tiny, the base model stays intact, and the result is a 46 MB adapter.

Q4. Explain the LoRA equation.

h = Wx + (α/r)·BAx. W is frozen; A (r×k) and B (d×r) are trainable with rank r = 16; α = 32 scales the update. B starts at zero so training begins from the original model.

Q5. What is the difference between LoRA and QLoRA?

Both train LoRA adapters on a frozen model. In QLoRA the frozen base model is stored in 4-bit NF4 instead of 16-bit, which cuts its memory about 4×. The adapters stay in bf16.

Q6. If the weights are 4-bit, how can training be accurate?

The weights are only stored in 4-bit; for every matrix multiplication they are dequantised to bf16. The base weights never change, so errors do not accumulate, and the adapters can compensate for quantisation error.

Q7. What is NF4? What is double quantisation?

NF4 is a 4-bit data type whose levels are optimal for normally distributed weights. Double quantisation also quantises the per-block scaling constants, saving about 0.4 bits per parameter.

Q8. Which layers did you apply LoRA to, and why?

q, k, v, o (attention) and gate, up, down (MLP) in all 28 layers. Adapting both attention and MLP layers generally gives better quality than attention alone.

Q9. How did you compute the trainable percentage?

PEFT's get_nb_trainable_parameters(): 18,464,768 trainable of 1,562,179,072 total = 1.18%. By hand: 659,456 LoRA parameters per layer × 28 layers.

Q10. Why compute the loss only on the response?

We want the model to learn to generate good answers, not to reproduce questions or the system prompt. Prompt tokens are masked with label −100.

Q11. How did you build the dataset, and is it good quality?

Hand-written structured content for 37 topics, converted by a generator into 14 task categories using the app's own templates, then cleaned, de-duplicated and split 90/10 stratified. The facts are consistent, but it is less diverse than a crowd-sourced dataset.

Q12. Why perplexity and not accuracy?

Many different explanations can be correct, so exact-match accuracy is meaningless for free text. Perplexity measures how likely the model finds the reference answers; side-by-side reading covers quality.

Q13. Is 71% lower perplexity proof that the model is better?

It proves that the model learned the target style and content of the dataset. It does not prove factual correctness — which is why I also compare answers qualitatively and report cases where the base model was better.

Q14. Did the model overfit?

Validation loss plateaued from step 175 while training loss kept falling — the start of memorisation — but validation loss never rose. Two epochs or early stopping would give the same result.

Q15. Why is the definition-category loss so low?

Definition text also appears inside other training examples for the same topic, so there is overlap between training and validation. A topic-level split would be a stricter test.

Q16. How does the app show base and fine-tuned answers without loading two models?

It loads the base model once, attaches the adapter with PEFT, and uses disable_adapter() for base answers. Same weights and same prompt, so the only difference is the adapter.

Q17. Why did you not merge the adapter?

Keeping it separate allows switching between base and fine-tuned for the comparison, keeps the file small (46 MB), and allows several adapters on one base model. Merging is optional and removes the tiny extra latency.

Q18. Why Qwen2.5-1.5B-Instruct?

Open licence, strong for its size, has a chat template, and fits QLoRA training on an 8 GB laptop GPU. It is configurable in config.py.

Q19. What are RLHF and DPO, and did you use them?

Both align models with human preferences after SFT. RLHF trains a reward model and optimises with PPO; DPO optimises directly on (chosen, rejected) pairs. This project uses only SFT; DPO is listed as future work.

Q20. What happens if CUDA runs out of memory?

The scripts catch the error and print which settings to reduce: batch size, sequence length, LoRA rank, 4-bit and gradient checkpointing, or a smaller model.

16. Limitations and Future Work

16.1 Limitations

- **Split by example, not by topic.** Validation examples share topics (and some text) with training examples, so perplexity overstates generalisation to new topics.
- **Generated dataset.** Categories for the same topic share facts; the data is consistent but less diverse than human-written data at scale.
- **Knowledge is unchanged.** ~750 examples teach style and format; the 1.5 B model can still make factual mistakes (e.g. flawed MCQ options).
- **Shorter answers.** The learned style is concise, which sometimes drops examples a beginner would find useful.
- **No human evaluation.** Teaching quality was not rated by students or teachers.

16.2 Future work

- Topic-level train/validation split and a separate held-out test set.
- Early stopping on validation loss (2 epochs appear sufficient).
- Preference tuning with **DPO** on (better answer, worse answer) pairs.
- Retrieval-augmented generation (RAG) over textbooks to reduce factual errors.
- A larger model (Qwen2.5-3B) and a larger, more diverse dataset.
- A human evaluation study with students.

17. Conclusion

EduTune AI shows the complete, genuine workflow of specialising a large language model: building an instruction dataset, fine-tuning with PEFT/LoRA on a 4-bit quantised base model (QLoRA), evaluating honestly, and deploying the result in a usable teaching application — all on a consumer laptop GPU in under 20 minutes of training.

By training only 1.18% of the parameters, the model learned a consistent teaching style and reliable output formats, and its perplexity on held-out teaching answers dropped from 8.612 to 2.482. The evaluation also makes clear what instruction tuning does *not* do: it shapes behaviour, but it does not guarantee factual accuracy — which is why the application carries a disclaimer and the report states its limitations openly.

Appendix A — Key Code

A.1 4-bit quantisation (utils/model_loader.py)

```
BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",           # NormalFloat4
    bnb_4bit_use_double_quant=True,      # quantise the scaling constants too
    bnb_4bit_compute_dtype=torch.bfloat16,
)
```

A.2 LoRA (training/train_qlora.py)

```
model = prepare_model_for_kbit_training(model, use_gradient_checkpointing=True)
lora_config = LoraConfig(
    r=16, lora_alpha=32, lora_dropout=0.05,
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj",
                    "gate_proj", "up_proj", "down_proj"],
    bias="none", task_type="CAUSAL_LM",
)
model = get_peft_model(model, lora_config)      # freeze base, add trainable A and B
trainable, total = model.get_nb_trainable_parameters()
```

A.3 Supervised fine-tuning (training/train_qlora.py)

```
SFTConfig(num_train_epochs=3, per_device_train_batch_size=2, gradient_accumulation_steps=4,
          learning_rate=2e-4, lr_scheduler_type="cosine", max_length=1024,
          completion_only_loss=True,           # loss on the response only
          bf16=True, gradient_checkpointing=True, optim="paged_adamw_8bit",
          eval_strategy="steps", eval_steps=25)
trainer = SFTTrainer(model=model, args=sft_config, train_dataset=ds["train"],
                     eval_dataset=ds["validation"], processing_class=tokenizer)
trainer.train()
trainer.model.save_pretrained("models/edu_tune_lora")   # adapter only
```

A.4 Base vs fine-tuned from one model (utils/model_loader.py)

```
model = PeftModel.from_pretrained(base_model, "models/edu_tune_lora")

with model.disable_adapter():      # temporarily = pure base model
    base_answer = model.generate(**inputs)
finetuned_answer = model.generate(**inputs)   # adapter enabled
```

Appendix B — Glossary

Term	Meaning
Adapter	Small set of extra trainable weights added to a frozen model (here: LoRA matrices)
bf16	bfloat16 — a 16-bit floating-point format with the range of fp32; used for computation
Chat template	The model's own format for system/user/assistant messages
Cross-entropy	Loss = −log probability of the correct token
Epoch	One full pass over the training data

Term	Meaning
Gradient accumulation	Summing gradients over several micro-batches before one update
Gradient checkpointing	Recomputing activations in the backward pass to save memory
LoRA	Low-Rank Adaptation: learn $\Delta W = B \cdot A$ with small rank r
NF4	4-bit NormalFloat data type used by QLoRA
PEFT	Parameter-Efficient Fine-Tuning: train few parameters, freeze the rest
Perplexity	e^{loss}; lower = the model finds the reference text more likely
QLoRA	LoRA on a 4-bit quantised, frozen base model
Quantisation	Storing numbers with fewer bits (here 16 → 4)
SFT	Supervised Fine-Tuning: training on examples with desired outputs
Token	A unit of text (word or sub-word) processed by the model
TRL	Hugging Face library for SFT, DPO and RLHF training